

Visual Voice Activity Detection (VVAD) com TinyML para suporte a pacientes traqueostomizados

Luiz Felipe Carneiro Pontes - 2023005638

Lucas Dias Franco Pereira - 2022015110

July 2, 2025

1 Introduction

A reconhecimento de fala visual (Visual Speech Recognition – VSR) é uma área emergente que busca interpretar a fala humana com base em imagens, normalmente focando na região da boca e mandíbula. Essa abordagem tem grande potencial para beneficiar pessoas com deficiências vocais severas, como aquelas submetidas a traqueostomia, que muitas vezes perdem completamente a capacidade de emitir sons audíveis. O artigo "Large Scale Visual Speech Recognition" mostra como sistemas baseados em VSR podem permitir que essas pessoas se comuniquem de forma mais eficiente, utilizando apenas a movimentação dos lábios para gerar textos ou comandos, ampliando significativamente sua autonomia e qualidade de vida.

No entanto, apesar do grande impacto potencial, os sistemas de VSR geralmente requerem modelos robustos, com alta capacidade de processamento e memória, o que dificulta sua aplicação direta em dispositivos embarcados de baixo consumo, como os utilizados em TinyML. Nesse contexto, o artigo "The VVAD-LRS3 Dataset for Visual Voice Activity Detection" propõe uma alternativa viável: a detecção de atividade vocal visual (Visual Voice Activity Detection – VVAD), que se limita a identificar quando uma pessoa está falando, sem necessariamente reconhecer o que está sendo dito.

Esse tipo de tarefa é consideravelmente mais leve computacionalmente e, portanto, mais compatível com as restrições impostas por plataformas embarcadas. Sistemas de VVAD podem funcionar como etapas preliminares de VSR ou, por si só, fornecerem feedback útil em contextos clínicos e assistivos. Ao adotar o VVAD como foco deste trabalho, foi possível explorar as contribuições da fala visual para pacientes traqueostomizados, respeitando as limitações de processamento, energia e memória típicas de sistemas TinyML.

2 Desenvolvimento

2.1 Sobre o dataset

O dataset utilizado neste projeto foi o VVAD-LRS3, uma adaptação do conhecido LRS3. Enquanto o LRS3 original era voltado para reconhecimento de fala visual, contendo anotações com as palavras pronunciadas em cada vídeo, o VVAD-LRS3 foi reformulado com o objetivo específico de detecção de atividade vocal. Em vez de identificar o que foi dito, o foco é em quando a pessoa está falando, com rótulos binários: True (falando) e False (não falando).

O VVAD-LRS3 contém diversos cliques de vídeo com rostos humanos capturados a 25 quadros por segundo (25 fps). Cada clipe é composto por 38 frames consecutivos, com resolução 96×96 pixels em RGB, resultando em uma curta sequência temporal para cada exemplo. Cada um desses cliques é rotulado com base na presença ou ausência de fala ao longo do trecho.

O dataset é organizado em quatro arquivos principais:

- faceImages: sequência de cliques com imagens completas dos rostos, com rótulo True ou False indicando se a pessoa está falando;
- lipImages: cliques focados apenas na região da boca, também rotulados com True ou False;

- faceFeatures: mapas de características extraídos dos rostos com o detector de landmarks faciais da biblioteca dlib;
- lipFeatures: similar ao anterior, mas restrito à região dos lábios.

Neste projeto, foi utilizado somente o arquivo faceImages, pois ele mantém a maior quantidade de informação espacial relevante para a análise da atividade vocal. No entanto, trata-se também do arquivo mais volumoso, com aproximadamente 35 GB de dados.

Para viabilizar o treinamento local, o dataset foi reduzido para cerca de 15 GB, divididos em três subconjuntos:

- Treinamento $\approx$ 5 GB
- Validação $\approx$ 5 GB
- Teste $\approx$ 5 GB

Além disso, o formato original .h5 foi convertido para .npy, o que tornou a manipulação mais simples e o treinamento mais rápido, especialmente ao empregar leitura com DataLoader em chunks — uma estratégia necessária para lidar com as limitações de RAM durante o processamento.

2.2 Pré-processamento

O pipeline de pré-processamento foi dividido em duas etapas principais: uma abordagem inicial para validação do funcionamento geral do sistema e, posteriormente, uma versão adaptada para os requisitos de TinyML e do hardware embarcado.

A primeira abordagem seguiu uma estrutura semelhante à descrita no artigo "The VVAD-LRS3 Dataset for Visual Voice Activity Detection". Foi implementado um DataLoader para carregar diretamente os cliques do arquivo faceImages (em RGB e resolução 96×96), mantendo a estrutura original dos dados: cliques com 38 frames. Essa versão visava validar a viabilidade do problema e do dataset.

O modelo treinado com essa abordagem robusta alcançou uma acurácia de validação máxima de 62%, servindo como referência de desempenho para versões posteriores mais leves.

Para tornar o sistema compatível com microcontroladores embarcados — como o Arduino Nano 33 BLE Sense, que utiliza a câmera OV7675 com taxa de captura limitada a cerca de 5 fps — foi necessário realizar uma série de adaptações nos dados.

Como a câmera embarcada trabalha em escala de cinza (grayscale), todos os cliques RGB foram convertidos usando a fórmula de luminância perceptiva:

$$\text{Gray} = 0.2989 \cdot R + 0.5870 \cdot G + 0.1140 \cdot B$$

Essa transformação reduz a dimensionalidade do dado e aproxima as condições de entrada daquelas que estarão disponíveis no dispositivo final.

Os cliques originais possuem 38 frames a 25 fps. Para simular as condições reais da câmera embarcada, os 38 frames foram divididos em 5 segmentos, cada um com aproximadamente 7 frames consecutivos ($38/5 \approx 7$). A ideia é representar visualmente o que a câmera embarcada capturaria em um segundo a 5 fps, sem perder completamente a variação temporal.

Cada imagem foi normalizada para o intervalo $[0, 1]$, e o batch size foi definido como 32, equilibrando desempenho e uso de memória.

2.3 Arquitetura do Modelo

Para viabilizar a implementação em ambientes restritos como microcontroladores (TinyML), foi projetada uma rede leve, dividida em duas partes principais: extração de características espaciais com CNN e modelagem temporal com LSTM, respeitando as limitações de RAM, armazenamento e velocidade de inferência.

O modelo recebe como entrada uma sequência de 7 quadros (frames), cada um com resolução de 96×96 pixels em escala de cinza. Essa estrutura foi escolhida para simular a taxa reduzida de amostragem (5 fps) da câmera embarcada OV7675, garantindo que o modelo reflita as limitações reais de aquisição de dados do hardware.

Cada um dos 7 frames é processado individualmente por uma pequena rede convolucional (CNN) aplicada de forma distribuída no tempo. Essa rede é composta por três camadas convolucionais intercaladas com camadas de pooling, aumentando gradualmente a profundidade dos filtros enquanto reduz as dimensões espaciais.

A última camada da CNN aplica uma média global sobre os mapas de ativação, transformando cada frame em um vetor fixo de 16 características. Isso permite capturar padrões visuais relevantes — como contornos da boca ou movimentos faciais — de forma eficiente e leve. A sequência de vetores extraída pelos frames é então processada por uma camada LSTM (Long Short-Term Memory), que aprende relações temporais entre os quadros. Essa etapa é fundamental para identificar o início, continuidade ou cessação do movimento da fala, já que a atividade vocal é inerentemente temporal.

A LSTM utilizada possui configuração simples, com ativação tangente hiperbólica e função sigmoide para o portão recorrente, além de ser "desenrolada" (unrolled), o que facilita sua conversão para o formato TFLite, que será usado posteriormente.

A saída temporal da LSTM é refinada por uma camada densa intermediária com ativação ReLU, seguida de uma camada final com ativação sigmoide. Essa última camada gera um valor entre 0 e 1 que representa a probabilidade da presença de fala no clipe de entrada, caracterizando uma tarefa de classificação binária.

2.4 Treinamento

O modelo foi treinado utilizando 10 épocas com o otimizador SGD, função de perda binária (binary crossentropy) e acurácia como métrica principal. O treinamento foi feito com um DataLoader otimizado para carregamento em chunks, e um callback de early stopping foi utilizado para evitar overfitting desnecessário caso a validação estabilizasse.

A validação apresentou um comportamento muito estável, com acurácia constante de 62,35% (val accuracy = 0.6235) ao longo de todas as épocas. Isso indica que o modelo atingiu um platô rapidamente, e que as alterações nos pesos durante o treinamento não trouxeram ganhos significativos após o primeiro ciclo.

Por outro lado, a acurácia de treinamento variou de forma mais ampla entre as épocas:

- Época 1: 44,31
- Época 3: Acurácia de 58,46
- Época 10: Cai para 45,83

Essa oscilação na acurácia de treino, sem impacto na validação, sugere que o modelo pode ter instabilidades na convergência ou estar em um regime de subajuste (underfitting). Ainda assim, a validação consistente em todas as épocas mostra que a rede aprendeu a detectar um padrão básico de atividade vocal. Um estudo mais aprofundado deverá ser feito.

2.5 Deploy

Devido às limitações dos microcontroladores embarcados — como restrição de memória RAM, armazenamento e suporte limitado a operações complexas — a estratégia de implantação do modelo final exigiu um desmembramento da arquitetura original em dois blocos funcionais.

- Bloco CNN (Convolucional): Responsável pela extração de características espaciais de cada frame da sequência. Esse bloco opera individualmente sobre os quadros de entrada e gera vetores compactos com informações discriminantes sobre os movimentos faciais.
- Bloco LSTM (Temporal): Recebe como entrada a sequência de vetores gerada pela CNN e aprende os padrões temporais — como a continuidade do movimento da boca ao longo do tempo — para decidir se há atividade vocal ou não.

Essa divisão é inspirada na própria natureza modular da arquitetura (uso de TimeDistributed) e permite otimizações independentes para cada componente, além de facilitar o debug e a conversão.

Cada bloco foi convertido separadamente para o formato .tflite e posteriormente para o .cc.

2.6 Implementação no Arduino

A implementação final foi realizada utilizando a placa Arduino Nano 33 BLE Sense com a câmera OV7675. O código embarcado foi escrito em C++ com a biblioteca TensorFlow Lite Micro, e foi estruturado em três blocos principais: captura e pré-processamento da imagem, inferência com a CNN, e inferência com a LSTM. O sistema opera em tempo real, analisando os frames capturados sequencialmente.

Durante a fase de inicialização, o código embarcado realiza a configuração da câmera OV7675, que opera no modo grayscale com resolução reduzida (160×120 pixels), adequada ao desempenho do microcontrolador. Em seguida, são carregados na memória os dois modelos .tflite — um para a CNN e outro para a LSTM —, cada um com sua própria região de memória alocada (tensor arena) de 50KB. O sistema também configura um pino de saída digital (o LED interno da placa) que será utilizado como sinalizador da detecção de fala.

O funcionamento principal ocorre dentro da função de repetição contínua (loop). A cada iteração, um novo frame é capturado pela câmera e redimensionado para 96×96 pixels por meio de uma interpolação simples (vizinho mais próximo). Este redimensionamento é necessário para compatibilizar a entrada da imagem com a dimensão esperada pelo modelo convolucional. Em seguida, os pixels da imagem são normalizados e convertidos para o formato int8, centrando os valores em torno de zero (por meio da subtração de 128), conforme especificado para modelos quantizados utilizados pelo TensorFlow Lite Micro.

Com a imagem devidamente preparada, ela é passada para a CNN, que realiza a inferência e gera um vetor de características reduzido, contendo 16 valores. Esse vetor representa os aspectos visuais mais relevantes da imagem e é armazenado em um buffer circular que guarda os últimos sete vetores extraídos — o equivalente a uma janela temporal de observação com cerca de 1,4 segundos de duração, considerando o tempo médio entre iterações.

Quando o buffer contém sete vetores completos, ele é concatenado em uma sequência única e enviado como entrada para o modelo LSTM. Essa rede recorrente processa a sequência de forma a capturar os padrões temporais da atividade facial observada ao longo do tempo, como mudanças na forma da boca ou movimentos repetitivos que indicam fala. A saída da LSTM é um valor numérico que representa a probabilidade de que o trecho analisado contenha atividade vocal.

Por fim, esse valor é interpretado como um sinal binário: se estiver abaixo de um determinado limiar, considera-se que a pessoa está falando; caso contrário, não há atividade vocal. Esse resultado é utilizado para acionar ou desligar o LED do Arduino, fornecendo um feedback visual simples, porém eficaz, sobre a detecção realizada.

Essa separação entre os modelos, além de modularizar a execução, permite maior controle sobre o uso de memória e o fluxo de dados, otimizando o desempenho geral e assegurando que o sistema funcione de forma estável e responsiva em um ambiente embarcado com recursos extremamente limitados.